



Department of Computer Science and Engineering

Academic Year 2025 – 2026 (Even Semester)

Degree, Semester & Branch: II Semester B.E. CSE 'B'

Course Code & Title: CS25C07 Object Oriented Programming

Name of the Faculty member (s): Dr.B.Vijayalakshmi, ASP-II/CSE

Innovative Practice Description

- **Unit / Topic:** Unit 4 / Compile Time versus Runtime Polymorphism

- **Course Outcome:** CO4

- **Activity Chosen:** Think Pair Share

- **Justification:**

Compile-time and runtime polymorphism are fundamental concepts in Object-Oriented Programming that enable flexibility and code reusability. Students often find it difficult to distinguish between method overloading (compile-time polymorphism) and virtual function-based overriding (runtime polymorphism). Think-Pair-Share (TPS) is an effective active learning strategy that encourages students to analyze, discuss, and compare these concepts collaboratively. Through peer interaction, students develop a deeper understanding of the role of pointers and virtual functions in achieving runtime polymorphism. The activity promotes critical thinking, communication skills, and conceptual clarity.

- **Time Allotted for the Activity:** 10 Minutes

- **Details of the Implementation:**

- The faculty member explained the concepts of compile-time polymorphism and runtime polymorphism with suitable C++ examples involving function overloading, function overriding, virtual functions, and base-class pointers.
- After the explanation, the Think-Pair-Share activity was conducted at the end of the session.
- Students are asked to form a team of two members and they were presented with the following question:
- “Consider a Shape base class with a virtual function display(). Circle and Rectangle are derived classes that override display(). Explain why a base-class pointer calling display() exhibits runtime polymorphism, whereas function overloading demonstrates compile-time polymorphism.”
- **THINK:** Students individually analyzed the question and identified the characteristics of both types of polymorphism.
- **PAIR:** Students paired with their classmates and discussed:
 - How function overloading is resolved during compilation.
 - How virtual functions are resolved during program execution.
 - The role of pointers in enabling dynamic binding.

- Students discussing compile-time and runtime polymorphism concepts during Pair phase is shown in Fig.1.
- **SHARE:** Each pair shared their observations with the class. Selected teams presented their understanding of the differences between static binding and dynamic binding, supported by examples. Students presenting their understanding of virtual functions and dynamic binding during Share phase in shown in Fig.2.
- ~~The faculty member consolidated the discussion and clarified misconceptions regarding polymorphism and virtual function mechanisms.~~
- **CO – PO / PSO mapping:**

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO4	3	3	3	1	1	-	-	1	1	1	1	3	3	3

(1 – Low 2 – Moderate 3 – High)

- **PO / PSO mapped:**

Innovative practice	PO1	PO2	PO3	PO9	PO10	PSO1
	3	3	3	1	1	3
Justification for correlation	Students apply programming fundamentals and object-oriented concepts to distinguish different forms of polymorphism.	Students analyze program behavior and identify the appropriate polymorphism mechanism for a given scenario.	Students develop solutions using virtual functions and inheritance to achieve dynamic behavior.	Students collaborate effectively with peers during pair discussions.	Students communicate technical concepts and reasoning during the sharing phase.	Students apply programming knowledge to solve object-oriented programming problems.

- **Images / Screenshot of the practice:**

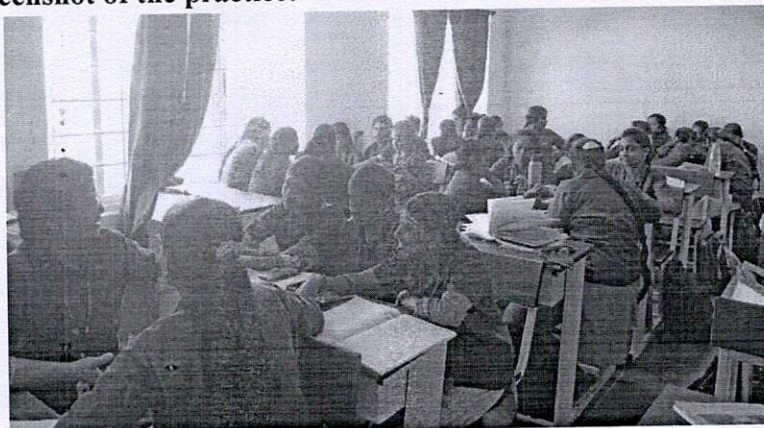


Fig 1. Students discussing during Pair phase



Fig 2. Students presenting their understanding of virtual functions during Share phase.

- **Reflective Critique:**

- ❖ **Feedback of practice from students and other stakeholders:**

- Students reported that comparing the two types of polymorphism through discussion improved conceptual understanding.
 - Students found peer interaction helpful in understanding the role of virtual functions and pointers.
 - Students expressed increased confidence in explaining static and dynamic binding concepts.

- ❖ **Benefit of the practice:** (E.g.: Outcome attainment would have increased due to innovative practice over conventional practice)

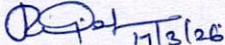
- Improved understanding of compile-time and runtime polymorphism.
 - Enhanced student engagement and participation in the classroom.
 - Encouraged collaborative learning and communication.
 - Strengthened problem-solving and analytical skills related to object-oriented programming.

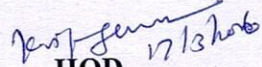
- ❖ **Challenges faced in implementation:**

- Some students initially confused function overriding with function overloading.
 - A few students required additional guidance to understand dynamic binding through base-class pointers.
 - Managing equal participation among all student pairs required continuous facilitation.

References:

- Think-Pair-Share Strategy –<https://www.readingrockets.org/classroom/classroom-strategies/think-pair-share>
- <https://cplusplus.com/doc/tutorial/polymorphism/>


Signature of Faculty Member


HOD